

Lattice Reptation Simulation: Indirect Test of Prediction 2

Summary of `prediction2.py`

(a) Theory

Prediction 2 and what it claims

Prediction 2 of the paper states that, under the expander assumption on the shape-class graph, the tube renewal time satisfies

$$\log \tau_{\text{renew}} \sim \alpha_0 \sqrt{L}, \quad \alpha_0 = \pi \sqrt{2/3} \approx 2.565$$

growing super-polynomially in L . This is qualitatively distinct from the standard Doi--Edwards reptation time $\tau_{\text{rep}} \sim L^3$, for which $\log \tau_{\text{rep}} = 3 \log L$ grows only logarithmically on a $\sqrt{L}$ axis. The physical origin is that the number of accessible tube topologies (integer partitions of L) grows as $\exp(\alpha_0 \sqrt{L})$ by the Hardy--Ramanujan formula, and the mixing time on an expander graph is proportional to the logarithm of the state space.

The crossover length

The two timescales are equal at a crossover length $L_{\times}$ where $\exp(\alpha_0 \sqrt{L_{\times}}) \sim L_{\times}^3$. A rough estimate gives $L_{\times} \sim 50$ -- 200 monomers. For $L \ll L_{\times}$ the chain is in the pre-asymptotic, reptation-dominated regime; for $L \gg L_{\times}$ the super-polynomial topology-renewal regime dominates. The accessible simulation range $L \leq 500$ (for reliable statistics) sits at or below this crossover, so the simulation is not expected to show the asymptotic $\exp(\alpha_0 \sqrt{L})$ scaling directly.

The observable and its limitations

The script measures the integrated autocorrelation time τ of the time series

$$C(t) = \mathbf{r}(0) \cdot \mathbf{r}(t)$$

where $\mathbf{r}(t)$ is the end-to-end vector at time t , as a proxy for τ_{renew} . This is an indirect test: the end-to-end vector decorrelates on the Rouse/reptation timescale, which equals the topology renewal time only in the asymptotic regime where topology renewal is the slowest mode. At accessible L the two timescales are not resolved, so the simulation tests the scaling of the effective relaxation time rather than τ_{renew} directly. A direct test would require tracking the autocorrelation of the run-length partition $\lambda(t)$ of the primitive path rather than the end-to-end vector.

The two competing fits

For each dataset the script fits two models to $\log \tau$ as a function of L :

- **Fit 1 (Prediction 2):** $\log \tau = a\sqrt{L} + b$, expected slope $a = \alpha_0 \approx 2.565$
- **Fit 2 (standard reptation):** $\log \tau = c \log L + d$, expected exponent $c = 3$

The diagnostic quantity is the ratio $\log \tau / \sqrt{L}$: if Prediction 2 holds this ratio should increase toward α_0 with L ; if a power law holds it decreases as $c \log L / \sqrt{L}$.

(b) What the script does

`prediction2.py` simulates standard lattice reptation for a single chain of length L on a 2D square lattice with periodic boundary conditions and a fixed obstacle field. For each L , it runs $n_{\text{chains}} \times n_{\text{runs}}$ independent realisations in parallel, estimates the mean and SEM of $\log \tau$ across realisations, fits both models above, and writes results to a CSV file.

The script accepts all run parameters via command-line arguments (chain lengths, lattice size, obstacle density, chain count, run count, seed, output filename) with modest defaults suitable for a quick sanity check. The production runs reported in the paper are invoked by `prediction2.sh`, which calls the script with six named configurations:

Run label	L values	ρ	Chains	Purpose
<code>simple</code>	20--200	0.05	32	Quick sanity check
<code>primary</code>	100--1000	0.05	128	Main scaling dataset
<code>density_low</code>	100--500	0.02	128	Density universality
<code>density_high</code>	100--500	0.10	128	Density universality
<code>seed_check_1</code>	200--500	0.05	128	Reproducibility (seed 100)
<code>seed_check_2</code>	200--500	0.05	128	Reproducibility (seed 200)

All production runs use a 2000×2000 lattice.

(c) How the script does it

Chain initialisation: loop-erased random walk

`grow_chain` places the initial chain using a loop-erased random walk (LERW). A simple random walker moves to a random obstacle-free neighbour at each step; if it revisits a site, the loop back to the previous visit is erased, keeping the path self-avoiding throughout. The LERW is guaranteed to produce a valid SAW on a well-connected lattice and runs in $O(L^2)$ expected time, in practice under 50ms for $L \leq 1000$ on a 2000×2000 lattice.

A greedy SAW (no backtracking, the naive alternative) fails completely for $L \geq 300$ on the obstacle lattice: the success rate per attempt falls to effectively zero because the walker traps itself irreversibly before completing the chain.

Reptation dynamics: slithering snake

`reptate_step` performs one reptation move. With equal probability the head or tail is selected as the active tip. A random obstacle-free and unoccupied neighbour of the tip is chosen; if one exists, the entire chain shifts one site in that direction, preserving chain length. Moves are rejected if no free neighbour exists. The set of

occupied sites is maintained incrementally (one addition and one deletion per accepted step), giving $O(1)$ cost per step.

End-to-end vector with PBC unwrapping

`end_to_end` computes the unwrapped end-to-end vector (r_x, r_y) by accumulating bond displacements with periodic boundary condition correction: each bond displacement is wrapped to $[-1, +1]$ if it exceeds 1 in magnitude.

Time series and autocorrelation time

`run_single_chain` records the time series $C(t) = r_x(0)r_x(t) + r_y(0)r_y(t)$, i.e. the dot product of the end-to-end vector at each thinned step with the initial end-to-end vector. The integrated autocorrelation time of this series is estimated by `integrated_autocorrelation_time` using the standard Madras--Sokal self-consistent windowing method with window constant $c = 5.0$: the estimator accumulates $\rho(t) = \text{cov}(t)/\text{var}$ at increasing lag t , stopping when $t \geq c\tau$. The returned τ is floored at 0.5 to avoid non-positive values.

The raw τ from the autocorrelation estimator is in units of thinned steps; it is multiplied by the thinning factor before being returned to give τ in units of sweeps.

Equilibration and sampling parameters

For each L , the equilibration and measurement lengths are set by:

$$\begin{aligned} \text{burn_in} &= \max(10L^2, 2\lfloor L^{2.5} \rfloor) \\ n_{\text{sweeps}} &= \min(\max(50L^2, 20\lfloor L^{2.5} \rfloor), 5,000,000) \\ \text{thin} &= \max(1, \lfloor L/5 \rfloor) \end{aligned}$$

The $L^{2.5}$ scale is a compromise between the L^2 Rouse scale and the L^3 Doi--Edwards scale. The 5,000,000-step cap means the thinned series covers approximately $n_{\text{sweeps}}/\text{thin}$ samples: around 100,000 at $L = 100$ falling to 25,000 at $L = 1000$. At $L = 750$ and $L = 1000$ the thinned series covers only $\sim 22\tau$ and $\sim 7\tau$ respectively, causing downward bias in the autocorrelation estimator; these points are excluded from quantitative fits in the paper.

Parallelisation and seeding

`measure_tau` constructs $n_{\text{chains}} \times n_{\text{runs}}$ independent jobs and distributes them across `multiprocessing.Pool` workers, with the worker count taken from `SLURM_CPUS_PER_TASK` if set. Each run index uses a distinct obstacle realisation (seed = `base_seed + run*1000`) and a distinct set of per-chain seeds (starting from `base_seed + run*1000 + 1`, incremented by 17 per chain), so all jobs are fully independent.

(d) Output produced

Primary run ($\rho = 0.05$, 128 chains, $L = 100\text{--}1000$)

L	$\sqrt{L}$	$\log \tau$	$\pm \text{SEM}$	$\log \tau / \sqrt{L}$
100	10.000	8.132	0.047	0.8132
150	12.247	9.065	0.013	0.7401
200	14.142	9.689	0.016	0.6851
300	17.321	10.496	0.026	0.6060
400	20.000	11.144	0.033	0.5572
500	22.361	11.572	0.038	0.5175
750	27.386	12.086	0.039	0.4413
1000	31.623	12.326	0.040	0.3898

Fit 1 (Prediction 2, $L = 100\text{--}500$): slope $a = 0.191$, $R^2 = 0.918$, $a/\alpha_0 = 0.074$.

Fit 2 (power law, $L = 100\text{--}500$): exponent $c = 1.87$, $R^2 = 0.998$.

The $L = 750$ and $L = 1000$ points are excluded from fits due to undersampling artefacts (see §(c) above).

Obstacle density runs

L	$\log \tau, \rho=0.02$	$\log \tau, \rho=0.05$	$\log \tau, \rho=0.10$	spread
100	8.137	8.132	8.248	0.116
200	9.645	9.689	9.728	0.083
300	10.515	10.496	10.651	0.155
500	11.497	11.572	11.594	0.097

Power-law exponent $c \approx 2.1$ in all three cases ($R^2 > 0.998$).

Seed reproducibility

L	seed 42	seed 100	seed 200	max diff
200	9.689	9.672	9.632	0.057
300	10.496	10.559	10.516	0.063
500	11.572	11.462	11.476	0.110

(e) What we learn

The simulation is in the pre-asymptotic, reptation-dominated regime throughout. The diagnostic ratio $\log \tau / \sqrt{L}$ decreases monotonically from 0.813 at $L = 100$ to 0.390 at $L = 1000$, reaching at most 32% of $\alpha_0 \approx 2.565$. If the super-polynomial regime had been reached, this ratio would be increasing toward α_0 . The power-law fit is decisively better at all L values with reliable statistics ($R^2 = 0.998$ vs 0.918 over $L = 100\text{--}$

500). This is entirely consistent with Prediction 2: the simulation operates in or just above the crossover zone, where a power law is the expected behaviour.

The fitted power-law exponent $c \approx 1.9\text{--}2.1$ is sub-Doi-Edwards. The Doi-Edwards prediction $c = 3$ applies to a 3D melt; the 2D obstacle lattice used here gives a lower effective exponent. The large- L collapse in $\log \tau / \sqrt{L}$ at $L = 750$ and $L = 1000$ is an artefact of the 5,000,000-step cap on n_{sweeps} : the thinned series covers only $\sim 22\tau$ at $L = 1000$ compared with $\sim 300\tau$ at $L = 150$, producing systematic downward bias in the autocorrelation estimator. These two points are excluded from quantitative fits.

The scaling exponent is robust to obstacle density. Across a fivefold range $\rho = 0.02\text{--}0.10$, the power-law exponent c and the values of $\log \tau$ agree to within 0.16 at fixed L . The simulation therefore characterises a regime that does not depend sensitively on the effective tube width set by obstacle density.

The between-realisation variance is comparable to the within-run SEM at large L . The three independent seeds agree in $\log \tau$ to within 0.06--0.11 (3--4 SEM), reflecting fluctuations in the obstacle environment rather than sampling error, and setting a floor on the achievable precision within a single obstacle realisation.

The observable is a proxy, not a direct measurement of τ_{renew} . The end-to-end vector decorrelation time and the tube topology renewal time coincide only when topology renewal is the slowest mode, i.e. deep in the super-polynomial regime $L \gg L_{\times}$. A definitive test of Prediction 2 requires either substantially longer runs at $L > L_{\times}$, or a simulation that directly tracks the autocorrelation of the run-length partition $\lambda(t)$ rather than the end-to-end vector. The present script establishes the simulation methodology and characterises the pre-crossover regime; it is presented as a feasibility test, not a verification of Prediction 2.